Varianta A

1. **Ce garantii ofera o bariera?**

   Tot codul dinainte de bariera este executat inainte de codul de dupa bariera.

2. **Cum poate scala o solutie paralelizabila cu 50% folosind legea lui Amdahl?**

   S = 1/(f + (1-f)/P) => S = 1/(1/2 + 1/2P) = 1/((1+P)/2P) = 2P/(1+P)
   P = 1 => S = 1
   P = 2 => S = 1.33
   P = 3 => S = 1.5
   P = 4 => S = 1.6
   P = 8 => S = 1.77

3. **Presudocod pentru Producator in cazul algoritmului Producator-Consumator.**

```
int N = MAX_BUF_SIZE;
int prim = 0;
int ultim = 0;
type buf[N];
sem gol = N;
sem plin = 0;
sem mutexP = 1;

Producator [i = 1…P] {
        while(true) {
                type elem = produce();
                P(gol);
                P(mutexP);
                buf[ultim] = elem;
                ultim = (ultim + 1) % N;
                V(mutexP);
                V(plin);
        }
}
```

4. **Implementare paralela pentru un joc de Sudoku.**

   ExecutorService. Se porneste in main cu configuratia initiala a tabloului de Sudoku si se trimite in ExecutorService. Functia din ExecutorService pune in coada de task-uri toate variantele posibile si corecte pornind de la configuratia primita. Intr-un final, cand se gaseste

solutie pentru tabloul Sudoku, se apeleaza functiile shutdown(), awaitTermination(time, measurementUnit), shutdownNow().

5. **Diferenta dintre ceas vectorial si ceas logic.**

   Ceasul logic este o singura valoare ce reprezinta cate evenimente (fie interne procesului, fie send sau receive) stie procesul curent ca s-au intamplat. In schimb, ceasul vectorial contine mai multe valori (egal cu numarul de procese) unde valoarea de pe pozitia i reprezinta prin cate evenimente stie procesul current ca a trecut procesul i.

6. **De ce algoritmul arbore este un algoritm unda?**

   Deoarece el indeplineste cele 3 conditii ale unui algoritm unda:
   a) Terminare – algoritmul se termina dupa un numar finit de pasi
   b) Decizie – algoritmul contine cel putin un eveniment decide
   c) Dependenta – fiecare decide este precedat cauzal de un eveniment in fiecare proces

7. **Rolul SHA-1 in algoritmul Chord.**

   SHA-1 asigura o distrubutie uniforma atat a identificatorilor cheilor, cat si a identificatorilor nodurilor.

8. **Ce garantie de consistenta (daca exista) e implementata in Blockchain?**

   Strong consistency. Trebuie ca toate nodurile sa stie noul lant de blocuri (cel mai lung).

Varianta B

1. **Descrieti taxonomia Flynn.**

   Sistemele pot fi grupate in 4 categorii:
   a) SISD – single instruction stream, single data stream (calculatorul classic single core)
   b) SIMD – single instruction stream, multiple data stream (placi video)
   c) MISD – multiple instruction stream, single data stream (sisteme specializate)
   d) MIMD – multiple instruction stream, multiple data stream (procesoare cu mai multe nuclee)

2. **Calculati speed-up dupa formula lui Amdahl cu un cod pentru care s-a reusit paralelizarea in proportie de 90%.**

   S = 1/(f + (1-f)/P) => S = 1/(1/10 + 9/10P) = 1/((9+P)/10P) = 10P/(9+P)
   P = 1 => S = 1
   P = 2 => S = 1.81
   P = 3 => S = 2.5
   P = 4 => S = 3.07
   P = 8 => S = 4.7

3. **Ce se intampla la rularea codului pentru algoritmul Producer-Consumer cu mai multi producatori si consumatori?**

   Cand un producator doreste sa adauge date in buffer, el scade valoarea semaforului “gol” (cel care permite sau nu scrierea in buffer), adauga datele in buffer si creste valoarea semaforului “plin” (cel care permite sau nu citirea din buffer). In mod analog, un consumator scade valoarea semaforului “plin” (pentru ca daca vrea sa citeasca, trebuie sa fie cel putin un element in buffer), citeste din buffer si creste valoarea semaforului “gol” (adica s-a eliberat o pozitie in buffer).

4. **Problema colorarii hartilor in paralel in 4 culori cu tari vecine care sa nu aiba aceeasi culoare.**

   ExecutorService. Se porneste de la a da unei tari o culoare anume si apoi functia run din ExecutorService va primi o configuratie si va adauga in coada de taskuri doar configuratii in care adauga o culoare astfel incat sa fie colorari partiale bune. Cand s-a gasit o solutie (s-au colorat toate tarile si regulile sunt respectate) se apeleaza metodele shutdown(), awaitTermination(time, measurementUnit), shutdownNow().

5. **Pd primeste un mesaj si vtm il pune in coada de intarziere si il livreaza doar daca …..**

   - vtm[s] = Vd[s] + 1 (este urmatorul timestamp pe care d il asteapta de la s)
   - vtm[k] <= Vd[k] pentru k != s (d a vazut toate mesajele ce au fost vazute de s la momentul cand a trimis mesajul

6. **Algoritmul ecou.**

   Procesul initiator trimite mesaje spre vecinii sai. Acestia trimit mesaje mai departe spre ceilalti vecini. In momentul in care un nod a primit mesaje de la toti vecinii sai, trimite un mesaj inapoi parintelui si poate decide. Intr-un final, initiatorul primeste mesaje de la toti vecinii si poate crea un eveniment decide.

7. Mesaje de confirmare versus mesaje cu marcaje.

La mesajele de confirmare, se realizeaza urmatoarele actiuni (pentru un arbore):

- Cand un process frunza termina, el isi anunta parintele
- Cand un nod oarecare termina, asteapta terminarea tuturor fiilor si apoi isi anunta parintele
- Notificarile de terminare se propaga pana la procesul initiator

In plus, daca este un graf dirijat aciclic:

- Deficitul unei legaturi = (nr mesaje de date transmise) – (nr semnale de confirmare primite)
- La terminare, un nod asteapta primirea semnalelor pe legaturile de iesire a datelor ca sa duca deficitul la 0 pe acele legaturi
- Tot la terminare, un nod trimite pe fiecare legatura de intrare a datelor un numar de semnale egal cu deficitul acelei legaturi

Daca graful are cicluri:

- Se genereaza un arbore de acoperire (fiecare nod isi salveaza ca “prim” primul nod de unde a primit mesaj (el fiind calea spre initiator)
- Pentru terminare, se trimit semnale pe toate legaturile de intrare (mai putin prim), apoi se receptioneaza semnale pentru toate legaturile de iesire, apoi se trimite semnal spre prim

La mesajele folosind marcaje:

- Procesele transmit mesaje de marcaj la sfarsitul comunicatiei
- Procesele primesc semnale de confirmare doar pentru marcaje
- Pentru terminare:
    - Se asteapta marcaje pe legaturile de intrare
    - Se transmit marcaje pe legaturile de iesire
    - Se transmit semnale pe legaturile de intrare (mai putin prim)
    - Se asteapta confirmare pe legaturile de iesire
    - Se transmite semnal pentru prim

Fiecare arc continue doua attribute: active si marcaj

Pentru fiecare arc de intrare(iesire) se inregistreaza daca:

- S-a primit(trimis) un mesaj (se pune activ pe true)
- S-a primit(trimis) un marcaj (se pune marcaj pe true)
- S-a primit(trimis) un semnal de confirmare (se pun activ si marcaj pe false)

8. Descrieti mecanismul de Blockchain forks.

Toate nodurile vor alege lantul de lungime mai mare. Daca un nod onest foloseste un blockchain de lungime N si este notificat de un blockchain de lungime M (M > N), nodul onest va comuta imediat pe folosirea acestuia. Tranzactiile din blocurile ramase “orfane” sun anulate.

Ca avantaje:

- toate nodurile bitcoin converg rapid catre un blockchain comun
- cu cat sunt mai multe blocuri adaugate pe chain, cu atat e mai improbabil ca tranzactiile noastre sa fie retrase
- blocurile extra servesc unor confirmari superioare (ele confirma validitatea tranzactiei)

Varianta C

1. **Ce garanteaza o zona critica? (o singura propozitie)**

   O zona critica garanteaza ca un singur fir de executie va executa codul din acea zona la un moment dat.

2. **Dati un exemplu de race condition, altul decat cel prezentat in curs. Explicati de ce este race condition.**

```
int x = 5;
int y;

T1
if (x == 5) {
        y = x * 2;
}

T2
x = 8;
```

   Posibile raspunsuri: y = 10 sau y = 16

3. **Ce se intampla cand se executa codul de mai jos?**

```
typeT buf[1:k];
int prim = 1, ultim = 1;
sem gol = k, plin = 0, mutex = 1;

process Producator[i = 1 to M] {
        typeT v;
        while(true) {
                v = produce();
                P(gol);
                   P(mutex);
```

```
                buf[ultim] = v;
                ultim = ultim mod k + 1;
            V(mutex);
        V(plin);
    }
}

process Consumator[i = 1 to N] {
    typeT w;
    while (true) {
        P(plin);
            P(mutex);
                w = buf[prim];
                prim = prim mod k + 1;
            V(mutex);
        V(gol);
        consuma(w);
    }
}
```

Raspuns: implementarea este foarte asemanatoare cu a unui producer-consumer cu mai multi producatori si mai multi consumatori si, desigur, un buffer cu mai mult de 1 element. Diferenta este ca in codul de mai sus bufferul poate fi accesat doar de un proces (fie consummator, fie producator) la un moment dat, deci un consummator nu poate scoate date in buffer in timp ce un producator adauga date in buffer si nici invers.

4. **Se da o lista de elemente (numere naturale). Scrieti pseudocod pentru o solutie paralela, eficienta, de a determina cea mai mica distanta intre oricare doua elemente.**

5. **Considerati o implementare de semafor distribuit cu doua tipuri de procese: procesele ce apeleaza operatiile semaforului (P si V) – Utiliz(i) si cele care implementeaza logica semaforului distribuit – Ajutor(i). Completati pseudocodul pentru procesele Ajutor in cazul receptionarii unui mesaj:**

```
while(true) {
    receive opsem[i](transm, k ,ts);
        // transm = ID sender; k = felul mesajului: P, V sau ack;
        // ts = timestamp-ul de trimitere
    TODO
    // sem = semafor
    // qm = coada de mesaje ordonata dupa ts
    cl = max(ts, cl) + 1;
    if (k == P or k == V) {
        insereaza (transm, k, ts) in locul corespunzator in qm;
```

```
                cl = cl + 1;
                broadcast opsem(I, ack, cl);
        } else if (k == ack) {
                inregistreaza transmiterea unui ack;
                for [mesajele V complet confirmate] {
                        scoate mesaj din qm;
                        sem = sem + 1;
                }
                for [mesajele P complet confirmate st sem > 0] {
                        scoate mesaj (transm, k, ts) din qm; //atentie la coliziuni
                        sem = sem – 1;
                        if (transm == i) {
                                cl = cl + 1;
                                send start[i](cl);
                        }
                }
        }
}
```

6. **Descrieti in pseudocod faza de wakeup pentru Algoritmul Arbore. Ce variabile foloseste fiecare process pentru aceasta faza?**

```
If (p este initiator) {
        ws = true;
        for [q = 1 to r st Vecini[q]] {
                wakeup[q]();
        }
}

while (wr < numar_vecini_p) {
        receive wakeup[p]();
        wr = wr + 1;
        if (NOT ws) {
                ws = true;
                for [q = 1 to n st Vecini[q]] {
                        send wakeup[q]();
                }
        }
}
```

Procesele folosesc variabilele:

- ws = un Boolean care asigura ca procesele trimit o singura data mesaje de wakeup
- wr = un intreg ce reprezinta cate procese vecine au trimit mesaj de wakeup procesului current

7. **Explicati ce reprezinta Finger Table in algoritmul Chord. Unde sta acest Finger Table?**

Un mechanism ce permite operatii de lookup in timp logN. Finger I redirecteaza catre succesorul lui n + 2^i. Exista un finger table in fiecare nod din reteaua inel peer-to-peer.

8. **Descrieti functionarea algoritmului Huang. Care sunt operatiile facute la trimiterea/receptionarea unui mesaj.**

   Se porneste cu un agent de control ce are initial ponderea 1. Toate celelalte procese au ponderea 0. Agentul de control trimite un mesaj de date unui process. Astfel, acel process devine activ si poate trimite mai departe mesaje de date.
   Operatii:
   - Send B(DW) – se imparte ponderea curenta W in doua ponderi W1 > 0 si W2 > 0 astfel incat W1 + W2 = W. Procesul current preia ponderea W1 si transmite ponderea W2
   - Receive B(DW) – se aduna ponderea primita la ponderea curenta: W_nou = W_vechi + DW. Daca nodul era liber, trece in stare active
   - Send C(DW) – se trimite intreaga pondere agentului de control si procesul devine liber
   - Receive C(DW) – doar agentul de control executa aceasta operatie. Aduna ponderea primita la ponderea curenta, iar daca este 1, declara “terminarea” algoritmului

Varianta D

1. **Definiti si dati formula pentru legea lui Amdahl.**

   Legea lui Amdahl spune ca speed-up-ul unui program paralel depinde de procentul de operatii din algoritmul secvential ce nu se pot executa in paralel.
   Sau
   Timpul de executie al unui program parallel depinde de procentul de operatii din algoritmul secvential ce nu se pot executa in parallel.

   Formula:
   $S = 1 / (f + (1 – f) / P)$
   Sau
   $T = fG + (1 – f) * (G / P)$

2. **Specificati macar un motiv pentru care ati regasi o situatie cand S > P.**

   Cand paralelizarea codului duce la impartirea memoriei folosite astfel incat aceasta sa se afle mai des in cache, lucru ce mareste considerabil executia programului. Programul secvential trebuia sa tot preia date din memoria RAM deoarece nu se putea salva pe toate in cache, in timp ce programul parallel are de cele mai multe ori (daca nu chiar de fiecare data) datele in

cache, facand de mult mai putine ori accese la memoria RAM decat algoritmul secvential. De aceea, intr-o situatie de acest fel se poate ca S > P.
O alta posibilitate este in rezolvarea problemelor prin algoritmi backtracking unde paralelizarea problemei poate dupa la gasirea mult mai rapid a unei solutii corecte.

3. **Dati un exemplu de situatie in Java cand folositi synchronized, si altul cand folositit wait/notify. Justificati utilitate folosirii unei constructii in detrimentul celeilalte.**

   Synchronized ca sa izolezi o zona critica. El deblocheaza singur accesul la zona critica ("da un unlock la final"). Wait si notify se folosesc intr-un bloc synchronized si ajuta la comunicarea intre threaduri deoarece daca un thread a ajuns la wait, el va fi pornit doar daca altul apeleaza notify.

4. **Parallel scan presudocod de corectat.**

   Lipseste bariera deoarece for-ul executa operatii pe elemente ce isi schimba valoarea de la un pas la altul si asta ar duce la rezultate aleatoare. Bariera trebuie pusa in for (fie prima linie din for, fie ultima).

5. **Lamport (relatia de "petrecut inainte")**

   Fie doua evenimente a si b:
   - Daca a si b sunt evenimente interne ale unui process si a il precede in timp pe b, atunci a->b
   - Daca a reprezinta un eveniment de transmitere a unui mesaj, iar b evenimentul de receptionare a aceluiasi mesaj, atunci a->b
   - Daca a->b si b->c, atunci a->c

6. **Caz favorabil pentru algoritmul LeLann de alegere a liderului.**

   Algoritmul LeLann classic nu are un caz favorabil anume (sau cel putin nu e prin curs), deci este vorba despre algoritmul LeLann-Chang-Roberts, unde inelul este "parcurs" (adica mesajele se transmit) in sensul acelor de ceasornic. Asta face ca aranjarea nodurilor in ordine crescatoare in sensul acelor de ceasornic sa dupa la cazul favorabil al algoritmului.

7. **Pseudocod algoritmul fazelor.**

   Curs 9, slide 34

8. **Mesaje de confirmare vs mesaje cu marcaje.**

   Varianta B, ex 7